Journal de bord

Mondes vivants · Sujet 04 ÉPHÉMÈRE · Simulation multi-agents du lancement d'une crème solaire fictive

> **À quoi sert ce document.** Après chaque étape du développement, on y explique ce qui a été fait, pourquoi, et ce qu'il faut retenir pour pouvoir le présenter au jury. Il se lit dans l'ordre : chaque étape s'appuie sur les précédentes. Les règles permanentes du projet sont dans `CLAUDE.md`, les décisions de conception dans `docs/contexte-projet.md`.

Le plan de travail

Le projet repart de zéro le 16 octobre 2026. Chaque étape est validée par le binôme avant de passer à la suivante, puis fait l'objet d'un commit. Les codes entre parenthèses renvoient aux exigences de l'énoncé.

#	Étape	Contenu	État
Phase 0 — Faire place nette			
0	Nettoyage du dépôt	Retrait du prototype non conforme, mise à jour de CLAUDE.md, README et journal	terminée
Phase 1 — Fondations			
1	Squelette du dépôt	Arborescence, `.gitignore`, `.env.example`, Python 3.12, ruff, fichier de configuration	terminée
2	Infrastructure Docker	Kafka (KRaft) et Redis en versions figées, création des 7 topics `sim.*`	terminée
3	Mesure de la machine	Réglages Ollama, benchmark par modèle en séquentiel puis en parallèle (E4)	terminée
Phase 2 — Le cœur, sans contenu du monde			
4	Contrat d'événements	Enveloppe Pydantic versionnée, deux horloges, `caused_by`, file `sim.dlq` (F4, C3, F6)	terminée
5	Horloge et orchestrateur	Mode synchrone, délai de livraison, `run_id` et graine (C2, C3)	terminée
6	Couche LLM	Sortie JSON imposée, routeur par palier, relance puis repli, incidents, coûts (A4, A5, E4)	terminée
7	Tranche verticale	3 agents LLM et 1 agent à règles sur 5 pas, lus par un client console	terminée
8	Historique et rejeu	Écriture Parquet, relecture d'un run sans appel LLM (E3, F5)	terminée
Phase 3 — Visualisation			
9	Pont FastAPI	WebSocket en direct ou en rejeu, inspection (D2), injection (D3)	terminée
10	Scène Godot 3D	Bâtiments distincts par zone, plage avec sable et mer, personnages animés, panneau d'indicateurs (D1)	à venir
Phase 4 — Le monde et ses agents			
11	Le monde	Nom de marque et test de contamination, population complète, seuils d'adoption	à venir
12	Corpus et RAG	Documents fictifs, ChromaDB, filtres par date et par rôle, mémoires, évaluation (F1, F2, A1, C6)	à venir
13	Agents à règles et baseline	Météo, vendeurs, concurrent, fidèles ; baseline C0 avec la même saisonnalité (A6, E1)	à venir
14	Indicateurs	Sens partagé, adoption, traçabilité, run témoin, taux de sortie de rôle (B1-B3, F6)	à venir
15	Mode asynchrone	Deuxième mode d'exécution, violations temporelles comptées (C4, C6)	à venir
Phase 5 — Preuves et soutenance			
16	Expériences	Configurations C0 à C3, au moins 5 runs chacune, Bass, synchrone/asynchrone, coûts (E1, E2, E4, E5, C5)	à venir
17	Charge et pannes	Agent muet, pic d'événements, latence jusqu'aux indicateurs (O4, C7)	à venir

18	Soutenance	Configuration de démo réduite, deux tailles de population (C8), README, run enregistré, rapport	à venir

Pourquoi cet ordre. On construit d'abord la tuyauterie (événements, horloge, appels LLM) avec très peu d'agents, pour vérifier que tout circule. Le rejeu arrive tôt (étape 8) parce qu'il permet ensuite de développer la scène 3D sur des runs enregistrés, sans occuper la carte graphique avec les modèles. Le contenu du monde (marque, corpus, population complète) vient une fois la chaîne fiable.

Étape 0 — Nettoyage du dépôt

16 octobre 2026 · Exigences concernées : aucune directement ; prépare toutes les suivantes.

Objectif

Repartir d'une base propre : ne garder dans le dépôt que ce qui est conforme aux règles du projet (`CLAUDE.md`), aux décisions de conception (`docs/contexte-projet.md`) et à l'énoncé.

Pourquoi on a repris de zéro

Un premier prototype avait été écrit avant que la conception soit arrêtée. En le comparant aux règles du projet, plusieurs écarts sont apparus, dont certains touchent des exigences notées :

Écart dans le prototype	Règle ou exigence concernée
Nom de marque, ingrédient et indice de protection écrits en dur dans le code et les prompts	Le nom doit rester un paramètre de configuration, choisi après un test de contamination
Topics `simulation.*` au lieu des 7 topics `sim.*` prévus	C1, architecture
Événements sans `run_id` , sans `caused_by` , et date de traitement en temps réel au lieu du temps simulé	C3 (deux horloges), F6 (traçabilité), E3 (rejeu)
Pas de Redis ; images Docker en version `latest`	F3 ; livrable reproductible sur une autre machine
Pas de `.gitignore` , fichiers compilés Python versionnés, arborescence différente	Conventions du dépôt
Visualisation 2D à base d'icônes	Remplacée par une scène 3D (décision du binôme)

Corriger ces écarts un par un aurait coûté plus cher que de reconstruire proprement, étape par étape.

Ce qui a été fait

- **Retrait du prototype du dépôt** : `src/` , `godot_client/` , `docker-compose.yml` , `requirements.txt` . Ces fichiers avaient été commités, ils restent donc consultables dans l'historique git (commit `607297b`).
- **Archivage hors du dépôt** des fichiers jamais commités, qu'une suppression aurait perdus définitivement : `src/agents/` , `src/rag/` , `src/simulation_engine.py` , `data/knowledge/` , les caches Godot et l'environnement virtuel. Ils sont dans `C:\Users\Siradiou\archive_prototype_S9_2026-10-16\` .
- **Mise à jour de `CLAUDE.md`** : chemin correct de l'énoncé, Python 3.12, Godot en 3D low-poly, et une section « Méthode de travail » qui fixe la règle d'avancement étape par étape.
- **Journal des décisions** complété dans `docs/contexte-projet.md` .
- **Création du `README.md`** : description du projet pour GitHub et tableau d'avancement, mis à jour à chaque commit.
- **Installation de Python 3.12.10** avec le gestionnaire d'installation officiel (`py install 3.12`), pour l'utilisateur seulement, sans droits administrateur. Python 3.14 reste installé et reste la version par défaut ; on appelle 3.12 explicitement avec `py -V:3.12` .
- **Création de ce journal de bord** : source HTML dans `docs/journal/journal-de-bord.html` , PDF généré à la racine par `docs/journal/build.ps1` (impression via Microsoft Edge, déjà présent sous Windows, donc aucune dépendance à installer).

Décisions prises à cette étape

Décision	Raison
Python **3.12**	Version stable bien prise en charge par les bibliothèques prévues (ChromaDB notamment), qui suivent souvent les nouvelles versions de Python avec retard. Python 3.12.10 a été installé à côté de 3.14 (voir ci-dessous).
Scène Godot en **3D low-poly**, rendu léger	Plus lisible et plus parlante pour le jury, tout en laissant de la mémoire graphique aux modèles LLM (budget d'environ 1 à 1,5 Go pour Windows et Godot).
Modèles 3D gratuits choisis sur maquettes	Plusieurs essais seront présentés au binôme, qui choisira (étape 10).
Avancement étape par étape, commit après validation	Chaque étape est comprise et expliquable par les deux membres avant d'aller plus loin.

État du dépôt après l'étape

```
CLAUDE.md                  règles permanentes du projet
README.md                  présentation et état d'avancement
journal-de-bord.pdf        ce document
docs/
  contexte-projet.md       décisions de conception
  Architecture_…pdf        schémas d'architecture
  Mondes-Vivants_…html     énoncé
  journal/
    journal-de-bord.html   source de ce document
    build.ps1              génération du PDF
```

À retenir pour l'oral

- Le projet a été reconstruit sur une conception écrite et validée avant de coder, pour que chaque composant réponde à une exigence identifiée de l'énoncé.
- Interpréteur et environnement virtuel sont deux choses distinctes : Python 3.12 est installé une fois sur la machine, puis l'environnement virtuel `.venv` du projet (étape 1) est créé à partir de lui et contient toutes les bibliothèques. Ainsi les dépendances du projet n'interfèrent pas avec le reste de la machine, et la procédure se reproduit sur la machine de la collaboratrice.
- Rien n'a été perdu : l'ancien code est dans l'historique git ou archivé.

Prochaine étape

Étape 1 — Squelette du dépôt : environnement virtuel `.venv` en Python 3.12, arborescence, `.gitignore`, `.env.example`, outillage ruff, et fichier de configuration qui portera `BRAND_NAME`, le nombre d'agents, le mode d'exécution et la graine.

Étape 1 — Squelette du dépôt

16 octobre 2026 · Exigences concernées : C2 (nombre d'agents fixé au lancement), livrable « reproductible sur une autre machine, versions figées ».

Objectif

Poser le cadre dans lequel tout le code viendra se ranger : un environnement Python isolé et reproductible, une arborescence qui suit l'architecture, des outils de qualité, et un fichier de configuration qui centralise tous les paramètres d'un run.

Ce qui a été fait

- **Environnement virtuel** `.venv` créé à partir de Python 3.12.10. Toutes les bibliothèques du projet y sont installées, rien sur le reste de la machine.
- **Arborescence** : un dossier par composant de l'architecture (`sim`, `agents`, `llm`, `bus`, `rag`, `indicators`, `bridge`, `replay`), plus `corpus`, `godot`, `configs` et `tests`. Chaque paquet Python porte une phrase qui dit à quoi il sert.
- `pyproject.toml` : fiche d'identité du projet. Il déclare les dépendances directes avec leur version exacte, configure ruff (style et erreurs courantes) et pytest.
- `requirements.lock` : liste complète et figée de toutes les bibliothèques installées, y compris les dépendances indirectes.
- `.gitignore` : exclut l'environnement virtuel, les caches, le fichier de secrets `.env`, les sorties de runs et le cache de Godot.
- `.env.example` : modèle du fichier de secrets, sans valeur (adresses d'Ollama, Kafka, Redis ; clés d'API facultatives).
- **Configuration** : `configs/default.yaml` et son chargeur `sim/config.py`, plus la liste des 11 rôles dans `agents/roles.py`.
- **11 tests automatisés** dans `tests/test_config.py`.

Décisions prises à cette étape

Décision	Raison
pip + fichier verrou	Outils standards de Python, rien à apprendre en plus. Le verrou fige aussi les dépendances indirectes : la collaboratrice obtient exactement les mêmes versions.
Configuration en YAML, validée par Pydantic	Lisible et commentable, prévu dans le document d'architecture. La validation transforme une faute de frappe en erreur claire au lancement plutôt qu'en comportement faux au milieu d'un run.
Une dépendance n'arrive qu'avec l'étape qui l'utilise	Chaque bibliothèque du projet est justifiée et expliquable. Ainsi `python-dotenv` (lecture de `.env`) attendra l'étape 2.
Nom de marque provisoire `MARQUE_PROVISOIRE`	Le vrai nom sera choisi à l'étape 11 après le test de contamination. Comme il n'existe que dans la configuration, le changer ne touchera aucun code.

Comment fonctionne la configuration

Un run est entièrement décrit par un fichier YAML en quatre sections :

Section	Contenu	Valeur par défaut
`run`	Graine du générateur aléatoire	42
`simulation`	Durée simulée en jours, mode d'exécution	90 jours, synchrone
`monde`	Nom de marque, ingrédient phare, indice de protection (fictifs)	valeurs provisoires
`population`	Nombre d'agents pour chacun des 11 rôles	120 agents, dont 84 LLM

Au chargement, `load_config` vérifie chaque valeur. Sont refusés : une durée nulle, une graine négative, un mode autre que `synchrone` ou `asynchrone`, un nom de marque vide, un rôle inconnu, un effectif négatif, une population vide, et toute clé mal orthographiée (par exemple `duree_jour` au lieu de `duree_jours`). L'erreur indique le fichier et le champ fautif.

> **Lien avec l'exigence C2.** Pour changer le nombre d'agents, on modifie la section `population` du fichier YAML, ou on crée un autre fichier dans `configs/`. Le code ne change pas. Un test le vérifie avec une population de 4 agents.

Les tests

Test	Ce qu'il garantit
`test_default_config_matches_design`	La configuration par défaut correspond à la conception : 120 agents, 90 jours, mode synchrone, les 11 rôles présents.
`test_agent_count_comes_from_config`	Le nombre d'agents vient bien du fichier (C2).
`test_invalid_values_are_rejected` (7 cas)	Chaque valeur invalide listée plus haut est refusée.
`test_empty_population_is_rejected`	Un run sans agent est refusé.
`test_missing_file_gives_clear_error`	Un fichier absent donne une erreur lisible.

Résultat : 11 tests réussis, aucune remarque de ruff. La procédure d'installation du README a été rejouée dans un environnement neuf pour vérifier qu'elle fonctionne telle qu'écrite.

État du dépôt après l'étape

```
pyproject.toml        identité du projet, dépendances directes, ruff, pytest
requirements.lock     toutes les versions figées
.gitignore  .env.example
configs/default.yaml  configuration principale (120 agents, 90 jours)
sim/config.py         chargement et validation de la configuration
agents/roles.py       les 11 rôles
tests/test_config.py  11 tests
sim/ agents/ llm/ bus/ rag/ indicators/ bridge/ replay/   paquets Python
corpus/ godot/        à remplir aux étapes 10 et 12
```

À retenir pour l'oral

- Un run est entièrement décrit par un fichier de configuration et une graine : c'est la base de la reproductibilité (E2, E3) et de l'exigence C2.
- La configuration est validée avant le lancement : une erreur de saisie ne peut pas fausser un run silencieusement.
- Les versions sont figées à deux niveaux (`pyproject.toml` pour les dépendances directes, `requirements.lock` pour toutes), ce qui répond à l'exigence de livrable reproductible sur une autre machine.

Prochaine étape

Étape 2 — Infrastructure Docker : Kafka en mode KRaft et Redis, versions figées, création des 7 topics `sim.*`, et vérification que Python peut y écrire et y lire.

Étape 2 — Infrastructure Docker : Kafka et Redis

16 octobre 2026 · Exigences concernées : C1 (événements transportés par Kafka), F3 (où vit chaque donnée), livrable reproductible.

Objectif

Mettre en place les deux services qui tournent à côté de la simulation : **Kafka**, le bus par lequel passent tous les événements, et **Redis**, qui garde l'état courant du monde. Vérifier que le code Python sait y écrire et y relire.

Rappel : à quoi servent ces deux services

Service	Rôle dans le projet	Pourquoi lui
Kafka	Journal ordonné de tous les événements : décisions des agents, messages, météo, indicateurs, incidents. Les agents ne s'appellent jamais directement, ils publient et lisent des événements.	Imposé par l'énoncé (C1). Garde l'ordre, permet de relire un flux, découple la simulation de Godot et des indicateurs.
Redis	État courant : stocks, météo, qui est dans quelle zone. Lu et écrit très souvent par les agents et par le pont vers Godot.	Base en mémoire, réponse en moins d'une milliseconde. Les données sont éphémères (F3).

Ce qui a été fait

- `docker-compose.yml` : décrit les deux conteneurs, avec des versions exactes (`apache/kafka:4.3.1`, `redis:8.8.3-alpine`) et un contrôle de santé pour chacun. `docker compose up -d --wait` les lance et attend qu'ils soient prêts.
- `bus/topics.py` : la liste des 7 topics sous forme d'énumération Python, et la commande `python -m bus.topics` qui crée ceux qui manquent. Elle peut être relancée sans risque : un topic déjà présent n'est pas recréé.
- `sim/settings.py` : lit les adresses des services dans `.env`. Sans fichier `.env`, les valeurs par défaut conviennent pour un poste local.
- **Trois bibliothèques ajoutées** : `aiokafka` (client Kafka compatible avec `asyncio`, que l'orchestrateur utilisera), `redis` (client Redis, avec une version `asyncio`) et `python-dotenv` (lecture de `.env`).
- **5 nouveaux tests**, dont 3 tests d'intégration qui parlent aux vrais services.

Les 7 topics

Topic	Ce qui y passe	Exigences
`sim.world.events`	Événements du monde : météo, stocks, promotions, événements injectés	C1, D3
`sim.agent.messages`	Échanges en langage naturel entre agents	A2
`sim.agent.decisions`	Action et sens déclarés par chaque agent	B1, D2
`sim.indicators`	Indicateurs calculés pendant le run	B2, F6
`sim.control`	Commandes et injections depuis l'interface	D3
`sim.incidents`	Sorties LLM invalides, replis, violations temporelles	A4, C6
`sim.dlq`	Événements qui ne respectent pas leur schéma (« dead letter queue »)	F4

Choix de configuration et leurs raisons

Choix	Raison
Kafka en mode **KRaft**, un seul nœud	Depuis Kafka 4, ZooKeeper n'existe plus : Kafka gère lui-même ses métadonnées. Un seul nœud suffit sur un portable.
Une partition par topic	Kafka ne garantit l'ordre qu'à l'intérieur d'une partition. Avec une seule partition, tous les événements d'un topic sont relus exactement dans l'ordre d'écriture, ce qui est indispensable pour rejouer un run à l'identique (E3). Le débit d'une partition dépasse de loin nos quelques dizaines de milliers d'événements par run.
Rétention de **7 jours**	Kafka sert de transport, pas d'archive. L'historique durable sera écrit en Parquet à l'étape 8 (F3).
Création automatique des topics **désactivée**	Par défaut, Kafka crée un topic dès qu'on écrit dedans. Une faute de frappe dans un nom créerait alors un topic fantôme où les événements se perdraient sans erreur. Désactivée, l'erreur est immédiate.
Mémoire de Kafka limitée à **1 Go**	Laisse la RAM aux autres composants. Mesuré au repos : 360 Mo pour Kafka, 5 Mo pour Redis.
Redis **sans persistance** sur disque	L'état courant est reconstruit à chaque run ; le sauvegarder ne servirait à rien et ralentirait les écritures.
Ollama **hors de Docker**	Sous Windows, donner le GPU à un conteneur ajoute de la complexité sans bénéfice (décision du document d'architecture).

Les tests

Test	Ce qu'il garantit
`test_all_topics_exist_with_expected_settings`	Les 7 topics existent, chacun avec une seule partition.
`test_kafka_roundtrip_preserves_order`	20 messages écrits dans un topic temporaire sont relus à l'identique et dans le même ordre ; le topic est supprimé ensuite.
`test_redis_roundtrip`	Une écriture dans Redis est relue correctement.
`test_defaults_match_env_example`	Sans `.env`, les adresses par défaut sont utilisées.
`test_env_file_and_environment`	`.env` est bien lu, et une variable d'environnement le remplace si elle est définie.

Résultat : 16 tests réussis, aucune remarque de ruff. On a aussi vérifié que, si les services sont arrêtés, les tests d'intégration sont ignorés avec le message « lancer `docker compose up -d` » au lieu d'échouer.

État du dépôt après l'étape

```
docker-compose.yml       Kafka 4.3.1 (KRaft) + Redis 8.8.3, versions figées
bus/topics.py            les 7 topics et leur création
sim/settings.py          adresses des services, lues dans .env
tests/test_infra.py      3 tests d'intégration (Kafka, Redis)
tests/test_settings.py   2 tests de lecture de .env
```

À retenir pour l'oral

- Kafka est le seul moyen de communication entre composants : c'est ce qui rend un run enregistrable, rejouable et observable sans modifier la simulation.
- Une partition par topic donne un ordre total des événements : c'est un choix délibéré au service du rejeu, pas une limite.
- Chaque stockage a un rôle et une durée de vie : Kafka transporte (7 jours), Redis garde le présent (le temps d'un run), Parquet gardera l'historique (étape 8).

Prochaine étape

Étape 3 — Mesure de la machine : vérifier Ollama et les modèles Qwen3, régler ses variables (modèles gardés en mémoire, requêtes parallèles), puis mesurer la latence de chaque modèle en séquentiel et en parallèle. Ces mesures remplaceront les durées supposées du document d'architecture et serviront de base au coût (E4).

Étape 3 — Mesure de la machine

16 octobre 2026 · Exigences concernées : E4 (coût d'un run), E5 (sobriété), A5 (modèles de tailles différentes), et l'objectif O4 (identifier le facteur limitant).

Objectif

Remplacer les durées *supposées* du document d'architecture par des durées *mesurées* sur la machine du projet, avant d'écrire le moindre agent. On sait ainsi combien de temps durera un run, combien de mémoire graphique prend chaque modèle, et ce que coûte le passage d'un palier à l'autre.

Ce qui a été fait

- **Téléchargement des trois modèles** Qwen3 : `qwen3:1.7b` (1,4 Go), `qwen3:4b` (2,5 Go), `qwen3:8b` (5,2 Go). Le premier réseau plafonnait à 140 Ko/s ; le changement de réseau a permis de finir à environ 10 Mo/s.
- **Réglage d'Ollama** par six variables d'environnement utilisateur (tableau ci-dessous), prises en compte au redémarrage d'Ollama.
- **Script de benchmark** `llm/benchmark.py`, relançable sur n'importe quelle machine (`python -m llm.benchmark`), y compris celle de la collaboratrice.
- **Résultats enregistrés** dans `docs/mesures/` : un résumé lisible (Markdown) et toutes les mesures brutes, appel par appel (JSON).
- **Bibliothèques ajoutées** : `ollama` (client officiel, qui servira aussi à l'étape 6) et `httpx` (requêtes HTTP simples).

Les réglages d'Ollama

Variable	Valeur	Raison
`OLLAMA_MAX_LOADED_MODELS`	3	Petit, moyen et modèle d'embeddings doivent rester chargés ensemble. Le document d'architecture proposait 2, mais le modèle d'embeddings compte aussi comme un modèle.
`OLLAMA_NUM_PARALLEL`	4	Jusqu'à 4 requêtes traitées en même temps par un modèle.
`OLLAMA_KEEP_ALIVE`	30m	Un modèle reste en mémoire 30 minutes après son dernier appel, pour éviter des rechargements.
`OLLAMA_FLASH_ATTENTION`	1	Calcul de l'attention plus rapide et plus économe ; nécessaire pour compresser le cache.
`OLLAMA_KV_CACHE_TYPE`	q8_0	Le cache de contexte est stocké sur 8 bits au lieu de 16 : deux fois moins de VRAM.
`OLLAMA_CONTEXT_LENGTH`	2048	Un prompt d'agent fait environ 650 tokens : 2 048 suffit et réserve moins de VRAM par requête.

Comment on a mesuré

Chaque appel imite une **décision d'agent** : un prompt d'environ 650 tokens (profil, jour, zone, météo, trois messages reçus, quatre passages « retrouvés »), une réponse JSON imposée (`action`, `sens`, `message`, `justification`), la réflexion de Qwen3 désactivée. Les prompts varient d'un appel à l'autre, mais de façon déterministe, pour que le benchmark soit reproductible. Le nom de marque vient de la configuration, jamais du code.

Pour chaque modèle : un chargement à froid, 12 appels l'un après l'autre, puis 12 appels par paquets de 2 et de 4 en parallèle. Pendant chaque série, `nvidia-smi` relève quatre fois par seconde la puissance, la mémoire et la température du GPU. On mesure enfin la **bascule de palier** : charger le 8b alors que le 1.7b et le 4b sont en mémoire, puis revenir au 1.7b.

Résultats

Mesure	qwen3:1.7b	qwen3:4b	qwen3:8b
VRAM occupée (modèle + cache pour 4 requêtes)	1,7 Go	3,1 Go	5,4 Go
Part du modèle sur le GPU	100 %	100 %	100 %
Latence médiane d'une décision, en séquentiel	0,76 s	1,49 s	2,14 s
Latence supposée dans le document d'architecture	0,8 s	1,5 s	3 s
Meilleur débit (décisions par minute)	108 (2 en parallèle)	78 (4 en parallèle)	58 (4 en parallèle)
Vitesse de génération en séquentiel	167 tokens/s	84 tokens/s	55 tokens/s
Énergie GPU par décision (séquentiel → meilleur parallèle)	56 → 40 J	109 → 56 J	167 → 79 J

Pendant le calcul, le GPU consomme environ 76 W et monte jusqu'à 80 °C (8b, 4 requêtes en parallèle).

Ce que les mesures nous apprennent

Constat	Conséquence
Les latences confirment les hypothèses, et le 8b est même plus rapide que prévu (2,1 s au lieu de 3 s).	Le dimensionnement tient. Pour les 2 155 appels estimés d'un run : environ 41 min en séquentiel, environ 25 min avec le meilleur parallélisme de chaque palier.
Le parallélisme double le débit du 4b et du 8b et divise par deux l'énergie par décision.	Argument direct pour la sobriété (E5) : regrouper les appels coûte moins cher.
La bascule de palier est coûteuse : charger le 8b décharge à la fois le 1.7b et le 4b (6,5 s), et revenir coûte environ 3 s par modèle. Un aller-retour ≈ 13 s.	Si le palier grand est appelé un jour sur deux, cela ajoute environ 10 minutes par run. L'ordonnancement par palier (étape 5) doit limiter le nombre de bascules.
Les réponses sont plus longues que prévu : environ 110 tokens au lieu de 70.	À maîtriser à l'étape 6 (justification plus courte).
Seules 2 réponses sur 117 sont invalides (1,7 %), et toutes pour la même raison : elles atteignent le plafond de 160 tokens et le JSON est coupé. Aucune réponse n'est hors schéma.	Le JSON imposé fonctionne même sur le 1.7b. À l'étape 6 : plafond relevé ou réponses raccourcies, et le mécanisme « une relance puis repli » (A4) couvrira le reste.
Le 1.7b à 4 requêtes simultanées est instable : la première vague d'appels prend 4 à 12 s avant de revenir à la normale (constaté deux fois).	Pour le palier petit, on visera 2 requêtes en parallèle ; 4 pour les paliers moyen et grand.
Les modèles ont une préférence : sur des situations variées, le 8b a répondu « tendance » 24 fois sur 38.	Point d'attention pour la question Q2 du jury : un sens partagé peut venir du modèle lui-même et non de la diffusion. Le run témoin de l'étape 14 devra le mesurer.

> **Premier chiffrage de la sobriété (E5), à confirmer à l'étape 16.** Avec la répartition prévue (270 appels au 8b, 715 au 4b, 1 170 au 1.7b), un run consomme environ 110 kJ côté GPU, soit 0,03 kWh, et environ 25 min de calcul. Les mêmes 2 155 appels tous sur le 8b demanderaient environ 170 kJ et 37 min. Ces chiffres ne comptent que le GPU pendant les appels, sans les bascules ni le reste de la machine.

État du dépôt après l'étape

```
llm/benchmark.py                            script de mesure, relançable
docs/mesures/benchmark_2026-10-16_1143.md   résumé des mesures
docs/mesures/benchmark_2026-10-16_1143.json  mesures brutes, appel par appel
tests/test_benchmark.py                     2 tests (prompt déterministe, taille)
```

Tests : 18 tests réussis, aucune remarque de ruff.

À retenir pour l'oral

- Le dimensionnement repose sur des **mesures**, pas sur des hypothèses : chaque chiffre renvoie à un fichier de `docs/mesures/`, appel par appel.
- Le facteur limitant est la **mémoire graphique** : les trois modèles ne tiennent pas ensemble, et changer de palier coûte environ 13 s. C'est ce qui justifie l'ordonnancement par palier.
- La mesure de l'énergie est en place dès maintenant : le coût de chaque run (E4) pourra être comparé à son apport (E1).

Prochaine étape

Étape 4 — Contrat d'événements : l'enveloppe commune à tous les événements (identifiant, version de schéma, `run_id`, deux horloges, `caused_by`), les schémas Pydantic de chaque type d'événement, et l'isolement des événements non conformes dans `sim.dlq`.

Étape 4 — Contrat d'événements

16 octobre 2026 · Exigences concernées : F4 (schémas documentés et versionnés, événements non conformes isolés), C1 (granularité justifiée), C3 (deux horloges), F6 (traçabilité), B1 (sens en catégories fermées).

Objectif

Fixer, avant d'écrire le moindre agent, la forme exacte de tout ce qui circule sur Kafka. Ce « contrat » est ce que la simulation, les indicateurs, l'historique, le rejeu et Godot ont en commun : s'il est clair et vérifié, chaque composant peut être écrit et testé séparément.

Ce qui a été fait

- `bus/events.py` : l'enveloppe commune et les 12 types d'événements, chacun avec son schéma Pydantic, sa version et son topic.
- `sim/domaine.py` : le vocabulaire fermé du monde (7 zones, 4 sens, 7 actions) et le temps simulé.
- `bus/producer.py` : publie un événement sur le topic de son type, avec accusé de réception et sans doublon.
- `bus/consumer.py` : lit et valide chaque message ; un message non conforme est envoyé dans `sim.dlq`, compté, et la lecture continue.
- `bus/export_schemas.py` : génère la documentation du contrat (`docs/evenements/README.md` et un JSON Schema par type) directement depuis le code. Un test échoue si la documentation n'est plus à jour.

L'enveloppe commune

Tout événement porte les mêmes champs, quel que soit son type :

Champ	Rôle	Exigence
`event_id`	Identifiant unique (UUID)	F6
`schema_version`	Version du schéma du type ; une version inconnue est rejetée	F4
`run_id`	Run qui a produit l'événement	E2, E3
`type`	Type d'événement, qui détermine la forme du `payload`	F4
`sim_time_occurred`	Moment *simulé* où le fait se produit	C3
`sim_time_processed`	Moment *simulé* où l'émetteur a fini de le traiter	C3
`wall_time_emitted`	Moment *réel* d'émission, en UTC	C3, E4
`source_agent_id`	Émetteur : un agent ou un composant	D2
`zone`	Zone concernée, si elle existe	D1
`caused_by`	Identifiants des événements à l'origine de celui-ci	F6, B4
`payload`	Contenu propre au type	—

> **Les deux horloges (C3).** Un agent déclenché le jour 3 à 10:00 (`sim_time_occurred`) dont la réflexion se termine à 10:20 simulées (`sim_time_processed`) produit un événement émis à 14 h 32 min 05 s, heure réelle (`wall_time_emitted`). Le contrat interdit qu'un fait soit traité avant de s'être produit. Un message porte en plus son moment de `livraison` : il n'est lisible qu'à partir de cette date, ce qui servira à détecter les violations de cohérence temporelle (C6).

Les 12 types d'événements

Topic	Types
`sim.world.events`	`horloge.pas` (début d'un jour), `monde.meteo`, `monde.stock`, `monde.perturbation`
`sim.agent.messages`	`agent.message` : direct, à toute une zone, ou sur les réseaux sociaux
`sim.agent.decisions`	`agent.decision` (action, sens, justification, porteur ou non, moteur, coût de l'appel LLM), `agent.deplacement`
`sim.indicators`	`indicateur.valeur`, avec sa fenêtre de calcul
`sim.control`	`controle.injection` (rupture de stock, polémique, vague de chaleur), `controle.commande` (pause, reprise, arrêt)
`sim.incidents`	`incident` : sortie invalide, hors rôle, sans réponse, repli, violation temporelle
`sim.dlq`	`dlq.rejet` : le message d'origine, son topic et la raison du rejet

Une décision indique aussi son **moteur** : `llm`, `regle` (agent à règles) ou `repli` (agent LLM retombé sur sa règle après un échec, A4). Une décision LLM porte obligatoirement le coût de son appel (modèle, tokens en entrée et en sortie, durée, nombre de tentatives) : le coût de chaque run (E4) se calculera directement à partir des événements.

Décisions prises à cette étape

Décision	Raison
Temps simulé en **minutes entières** depuis le jour 1 à 00:00	Un pas dure un jour, mais un message peut être émis le soir et lu le lendemain midi (exemple de l'énoncé pour C3). Des entiers se comparent exactement, sans erreur d'arrondi : indispensable pour rejouer un run à l'identique et détecter une violation temporelle.
Un événement par fait observable (granularité, C1)	Une décision, un message, un déplacement, un changement de météo ou de stock. Jamais un instantané complet de l'état à chaque pas : ce serait volumineux et redondant. C'est la plus petite unité qui suffit au rejeu et à la traçabilité.
7 actions fermées : acheter, appliquer, recommander, critiquer, en parler, ignorer, abandonner	Comme pour le sens, une liste fermée évite le texte libre, raccourcit les réponses des petits modèles et rend les décisions comptables. On pourra l'étendre en incrémentant la version du schéma.
JSON validé par Pydantic plutôt qu'Avro avec un registre de schémas	Lisible à l'œil, natif en Python, sans serveur supplémentaire. Les JSON Schema exportés jouent le rôle de documentation formelle.
Documentation **générée depuis le code**	Elle ne peut pas être fausse : un test vérifie qu'elle correspond au code.
Champs **obligatoires à la réception**	À la construction, l'identifiant et l'horodatage sont remplis automatiquement. Mais un message *reçu* sans identifiant est rejeté : en inventer un casserait la traçabilité.

Ce que le contrat refuse

En plus des types de champs, des règles de cohérence sont vérifiées à chaque validation :

- un fait traité avant de s'être produit, ou un événement qui se cite lui-même comme cause ;
- un sens, une action ou une zone hors des listes fermées, un champ inconnu ;
- une version de schéma inconnue, un horodatage sans fuseau horaire ;
- un agent à règles qui déclarerait un palier ou un appel LLM ; une décision LLM sans son coût ;
- un message livré avant d'avoir été émis, un message direct sans destinataire ;
- un déplacement qui ne change pas de zone, un stock sans zone.

Les tests

Test	Ce qu'il garantit
Aller-retour des 12 types	Chaque type s'encode puis se relit à l'identique, et part sur le bon topic.
15 décisions invalides + 5 messages illisibles	Chaque cas de la liste ci-dessus est refusé, avec un message qui nomme le champ fautif.
Couverture des topics	Chacun des 7 topics transporte au moins un type ; aucun nom de type n'est en double.
Documentation à jour	La documentation générée correspond exactement au code.
Intégration : `sim.dlq`	Sur un vrai Kafka, 4 messages dont 2 invalides (texte illisible, version inconnue) : les 2 valides sont lus dans l'ordre, les 2 autres arrivent dans la file de rejet avec leur raison, et la lecture ne s'arrête pas.

Résultat : **61 tests réussis** au total, aucune remarque de ruff.

État du dépôt après l'étape

```
sim/domaine.py            zones, sens, actions, temps simulé
bus/events.py             enveloppe et 12 types d'événements
bus/producer.py           publication
bus/consumer.py           lecture, validation, file de rejet
bus/export_schemas.py     génération de la documentation
docs/evenements/          README.md + 12 JSON Schema (générés)
tests/test_events.py      42 tests unitaires du contrat
tests/test_bus.py         test d'intégration de sim.dlq
```

À retenir pour l'oral

- Le contrat d'événements est le point de rencontre de tous les composants : simulation, indicateurs, historique, rejeu et Godot ne se connaissent pas, ils partagent ce contrat.
- Un message non conforme ne fait jamais tomber la simulation : il est isolé dans `sim.dlq` avec sa raison, et compté.
- `caused_by` relie chaque événement à ses causes : c'est ce qui permettra de remonter d'une valeur d'indicateur aux décisions et messages qui l'ont produite (F6, B4).

Prochaine étape

Étape 5 — Horloge simulée et orchestrateur en mode synchrone : faire avancer le temps jour par jour, déclencher les agents, appliquer le délai de livraison des messages, identifier chaque run par un `run_id` et une graine.

Étape 5 — Horloge simulée et orchestrateur (mode synchrone)

17 octobre 2026 · Exigences concernées : C2 (population fixée au lancement), C3 (horloge simulée explicite), C4 (mode synchrone), C6 (pas d'information du futur), A4 (un agent en panne n'arrête pas le monde), E2 et E3 (reproductibilité), F3 (état courant dans Redis).

Objectif

Construire le moteur de la simulation : ce qui fait avancer le temps, décide quels agents réfléchissent, dans quel ordre, avec quelles informations, et publie le résultat sur Kafka. Les vrais agents LLM n'existent pas encore : on teste le moteur avec des **agents factices**, qui respectent la même interface et imitent la latence mesurée des modèles.

Ce qui a été fait

Fichier	Rôle
`sim/horloge.py`	Horloge simulée : elle n'avance que sur ordre de l'orchestrateur, jamais à reculons ; ordre de passage des paliers.
`sim/livraison.py`	Boîte aux lettres : un message n'est remis qu'une fois livré.
`sim/orchestrateur.py`	Le déroulé d'une journée, de l'horloge à la publication.
`sim/etat.py`	Écriture de l'état courant dans Redis à la fin de chaque jour.
`sim/run.py`	Identité du run (`run_id` , graine, manifeste) et commande de lancement.
`agents/base.py`	Interface commune des agents : perception, décision, déclencheur.
`agents/factices.py`	Agents et météo factices, sans LLM, pour tester le moteur.
`agents/roles.py`	Attribution du palier selon le rôle (tableau A5 de `CLAUDE.md`).

Le déroulé d'une journée

1. **00:00** : l'horloge publie le début du jour, puis les acteurs du monde publient leurs événements (météo…).
2. **10:00** : chaque agent reçoit sa **perception**, construite sur l'état de *la veille* : sa zone, la météo, la part de porteurs autour de lui, les messages qui lui ont été livrés, les perturbations survenues.
3. Seuls les agents **déclenchés** réfléchissent : premier jour, message reçu, ou perturbation. Les autres gardent leur décision précédente, sans aucun appel (règle 10).
4. Les agents à règles décident d'abord, puis les agents LLM **regroupés par palier**, avec le nombre de requêtes simultanées fixé par la configuration.
5. Publication, **dans l'ordre des identifiants d'agents** : décisions, puis déplacements et messages qui en découlent.
6. Écriture de l'état courant dans Redis.

> **La barrière synchrone.** Le jour *d*, chaque agent ne voit que l'état du jour *d-1*. L'ordre dans lequel les agents réfléchissent pendant la journée n'a donc aucun effet sur leurs décisions. C'est ce qui permet de regrouper librement les appels par modèle sans fausser le phénomène, et c'est ce qui distingue le mode synchrone du mode asynchrone (étape 15), où les agents rapides agiront avant les lents.

Moins de bascules entre modèles

Le grand modèle ne tient pas en mémoire avec les deux autres : un aller-retour coûte environ 13 s (étape 3). L'ordre des paliers **alterne d'un jour à l'autre** : petit → moyen → grand les jours impairs, grand → moyen → petit les jours pairs. Deux jours consécutifs se touchent par le même modèle, déjà chargé.

Ordre	Bascules sur 6 jours (mesuré par un test)
Toujours petit → moyen → grand	11
Alterné	**6**, soit une par jour

Sur un run de 90 jours, cela représente environ 90 bascules évitées, soit une dizaine de minutes.

Le temps et les messages

Un message émis à 10:00 avec un délai de livraison δ est lisible à partir de 10:00 + δ. Les délais sont des paramètres de la configuration :

Canal	Délai	Destinataires
Zone	immédiat	les agents présents dans la zone en fin de journée
Direct	2 h	les agents nommés
Réseaux sociaux	6 h	toute la population (provisoire, voir plus bas)

En mode synchrone, avec la barrière, tout délai inférieur à 24 h donne une lecture le lendemain ; un délai de 30 h, le surlendemain. Les tests vérifient ces cas un par un.

Cohérence temporelle (C6) et pannes (A4)

- **Par construction**, un agent ne reçoit que les messages déjà livrés.
- **Par contrôle**, si une décision cite (`caused_by`) une information que l'agent n'avait pas reçue, l'orchestrateur publie un incident `violation_temporelle`, et le compte. Un test utilise volontairement un agent « tricheur » pour le prouver.
- Un agent qui plante ou ne répond pas produit un incident `sans_reponse` et garde sa décision précédente : la journée continue.
- Une décision mal formée (message trop long…) produit un incident `sortie_invalide` et rien d'autre : jamais une décision publiée à moitié.

Identité et reproductibilité d'un run

- Chaque run reçoit un identifiant lisible : `20261017-173539-default-s42` (date, configuration, graine) et un dossier `runs/<run_id>/` avec un **manifeste** (configuration complète, graine, versions, commit git) et un **bilan** (compteurs).
- Chaque agent a son propre générateur aléatoire, dérivé de la graine du run et de son identifiant : ses tirages ne dépendent pas de l'ordre de passage des autres.

Deux défauts trouvés et corrigés grâce aux tests

Défaut	Correction
Deux runs de même graine **divergeaient dès le jour 2** : les messages reçus le même jour étaient départagés par leur identifiant, tiré au hasard à chaque run.	Tri par moment de livraison puis ordre de dépôt, qui est déterministe. Un test compare maintenant deux runs lancés dans **deux processus distincts**, pour détecter aussi les dépendances à l'ordre interne de Python.
La lecture de `.env` **écrivait** ses valeurs dans l'environnement du processus : un test y laissait une fausse adresse Redis, et un test d'intégration lancé après était ignoré sans échouer.	Le fichier est désormais lu sans rien modifier. Un test le vérifie.

Ce que montrent les premiers runs

Run factice de 5 jours avec 119 agents (la météo est un acteur du monde, pas un agent) : 595 décisions, 186 messages, 157 déplacements, 0 incident, 0 violation temporelle, en moins d'une seconde. Avec les latences mesurées divisées par 10 : 11,7 s pour 3 jours, conforme au calcul (environ 3,3 s par jour).

> **Constat important pour le budget.** Aucune décision n'est jamais « conservée » : un message sur les réseaux sociaux est aujourd'hui livré à *toute* la population, si bien que chaque agent reçoit un message chaque jour et réfléchit chaque

> jour. Avec de vrais modèles, cela ferait 84 × 90 = **7 560 appels** par run (environ 50 min), au lieu des quelque 2 155 prévus. À l'étape 11, la diffusion sur les réseaux sera limitée aux **abonnés** de l'émetteur.

Les tests

26 nouveaux tests : horloge, alternance des paliers et nombre de bascules, limite de parallélisme, ordre de publication indépendant des latences, délais de livraison (4 cas), barrière synchrone, déclencheurs, violation temporelle, citation autorisée d'un message reçu, agent en panne, décision mal formée, reproductibilité (même processus et deux processus), population factice, configuration invalide, et un run complet de 3 jours sur le vrai Kafka et le vrai Redis.

Résultat : **87 tests réussis**, aucun ignoré, aucune remarque de ruff.

À retenir pour l'oral

- Le temps simulé est un objet explicite, piloté par l'orchestrateur : il ne dépend jamais de la vitesse des modèles en mode synchrone.
- La barrière synchrone rend l'ordre de passage sans effet, ce qui autorise le regroupement par palier et divise par deux les bascules de modèle.
- Un run est reproductible : même configuration et même graine donnent les mêmes décisions, ce qui est vérifié automatiquement.
- Les déclencheurs sont le premier levier de sobriété : sans eux, le coût d'un run triple.

Prochaine étape

Étape 6 — Couche LLM : client Ollama avec sortie JSON imposée et réflexion désactivée, routage vers le bon modèle selon le palier, une relance puis repli sur une règle en cas de réponse invalide, incidents publiés, coût de chaque appel journalisé.

Étape 6 — Couche LLM

18 octobre 2026 · Exigences concernées : A4 (tolérance aux pannes d'agent), A5 (modèle selon le rôle), B1 (sens en catégories fermées), E4 (coût de chaque appel), F6 (traçabilité), règle 5 de `CLAUDE.md` (sortie structurée).

Objectif

Faire décider de vrais agents par les modèles Qwen3 : leur donner un prompt construit à partir de leur perception, obtenir une réponse dont la forme est garantie, savoir quoi faire quand le modèle échoue, et mesurer ce que coûte chaque décision. Les agents LLM sont prêts ; ils seront branchés dans un vrai run à l'étape 7.

Ce qui a été fait

Fichier	Rôle
`llm/sortie.py`	Schéma JSON imposé au modèle et lecture validée de sa réponse.
`llm/client.py`	Appel à Ollama : JSON imposé, réflexion désactivée, graine, mesure des tokens et de la durée.
`llm/routeur.py`	Quel modèle pour quel palier, délai maximal, préchargement du modèle avant chaque groupe.
`agents/agent_llm.py`	L'agent LLM : prompt, appel, une relance, puis repli sur règle.
`agents/prompts.py`	Prompts (rôle, produit, situation du jour), construits depuis la configuration.
`agents/regles.py`	Seuil d'adoption individuel et règle de repli.
`llm/validation.py`	Mesure de la couche sur les vrais modèles (`python -m llm.validation`).
`sim/orchestrateur.py`	Publie les incidents de chaque décision et totalise les coûts par modèle.

Décisions prises à cette étape

Question	Décision	Raison
Règle de repli quand le modèle échoue deux fois	**Seuil + continuité** : l'agent garde son avis de la veille ; s'il n'en a pas, il adopte si la part de porteurs autour de lui atteint son seuil individuel	Un repli ne doit pas changer brutalement le sens d'un agent. La règle de seuil est celle de Granovetter, réutilisée par les agents à règles (étape 13).
Ce que le modèle décide dans sa réponse	Justification, sens, action, et un message facultatif avec son canal (zone, direct à un contact, réseaux)	Le canal fait partie de la décision de parler. Les déplacements restent gérés par des règles (étape 11) : un champ de plus coûterait des tokens et des erreurs au petit modèle.
Porteur ou non	Déduit de l'action : acheter ou appliquer fait un porteur, abandonner l'en retire, le reste ne change rien	Une règle simple et explicable, plutôt qu'un champ de plus à remplir par le modèle.
Modèle de chaque palier	Dans la configuration (`llm.modeles`)	Les configurations C2 (tout sur le 8b) et C3 (tout sur le 1.7b) se lancent sans toucher au code.
Réglages des appels	Sortie plafonnée à 320 tokens, délai maximal 60 s, température 0,8	À 160 tokens, l'étape 3 voyait des JSON tronqués. 60 s laisse de la marge au 8b chargé à froid.

Une réponse dont la forme est garantie

Le schéma JSON est donné au paramètre `format` d'Ollama, qui **contraint la génération** : le modèle ne peut écrire qu'un JSON de cette forme. Le sens et l'action ne peuvent prendre que les valeurs des catégories fermées (B1). Le schéma est propre à chaque agent : un message direct ne peut viser qu'un de *ses* contacts.

```
{ "justification": "…une ou deux phrases…",
  "sens": "efficace | tendance | banal | depasse",
  "action": "acheter | appliquer | recommander | critiquer | en_parler | ignorer | abandonner",
  "message": null
           | { "canal": "zone | reseaux", "contenu": "…" }
           | { "canal": "direct", "destinataire": "<un contact>", "contenu": "…" } }
```

La réponse est ensuite **revalidée** par Pydantic, car la contrainte d'Ollama ne couvre pas tout (texte vide, champ en trop…).

> **Pourquoi la justification vient en premier.** Ollama écrit les champs dans l'ordre du schéma. En plaçant la justification avant le sens et l'action, le modèle formule sa raison *avant* de choisir, au lieu de justifier après coup un choix déjà fait.

Un essai préalable a montré que la longueur maximale imposée par le schéma **coupe le texte en plein mot**. Elle ne sert donc que de garde-fou (300 caractères) ; c'est le prompt qui demande des réponses courtes.

Quand le modèle échoue : relance, puis repli (A4)

1. Réponse invalide ou délai dépassé : un incident (`sortie_invalide` ou `sans_reponse`), puis **une relance**, qui rappelle au modèle la raison du refus.
2. Second échec : incident `repli`, et la décision vient de la règle « seuil + continuité ». Elle est publiée avec `moteur = repli` : on sait toujours quelles décisions ne viennent pas du modèle.

L'agent ne publie rien lui-même : il rend ses incidents avec sa décision, et l'orchestrateur les publie sur `sim.incidents`, rattachés à la décision par `caused_by` (F6). Une décision cite dans `caused_by` exactement les messages montrés au modèle : au plus les 8 plus récents, pour tenir dans le contexte de 2048 tokens.

Mesurer le coût (E4)

- Chaque décision porte son coût : modèle, tokens en entrée et en sortie, durée, nombre de tentatives. Les tentatives ratées sont comptées : elles ont consommé du calcul.
- Le bilan du run totalise appels, relances, replis, tokens et durée, **par modèle**.
- Avant chaque groupe de palier, le routeur **précharge le modèle** : le temps de bascule est mesuré à part au lieu d'alourdir la latence du premier agent.

Résultats sur les vrais modèles

72 décisions (24 par modèle, un agent par rôle, situations variées), avec le parallélisme de la configuration. Détail dans `docs/mesures/couche-llm_2026-10-18_1131.md`.

Modèle	Valides au 1er coup	Replis	Latence médiane	p90	Tokens entrée / sortie	Messages envoyés
qwen3:1.7b	24/24	0	0,9 s	1,0 s	581 / 105	0/24
qwen3:4b	24/24	0	2,5 s	4,0 s	580 / 117	7/24
qwen3:8b	24/24	0	4,0 s	7,5 s	592 / 119	24/24

Chargement du 8b avant son groupe : 7,4 s. Aucun texte hors français dans cette série (un essai préalable du 1.7b avait laissé passer un caractère chinois : la mesure le compte désormais).

> **Constats à garder pour la suite.**
>
> - **La propension à parler dépend de la taille du modèle** : le 8b envoie un message à chaque décision, le 1.7b jamais. C'est un effet du modèle, pas du rôle : il pèse sur la diffusion et sur la comparaison demandée par A5. À traiter à l'étape 11 (prompts par rôle, budget de messages) et à mesurer à l'étape 14.
> - Le 1.7b produit parfois une action incohérente avec son sens (« banal » mais « acheter ») et des justifications de trois phrases. Ce sera l'objet du taux de sortie de rôle (étape 14).
> - La graine rend chaque appel reproductible pris isolément, mais le traitement en lot d'Ollama peut faire varier une réponse. Le rejeu exact d'un run passera donc par son journal (étape 8), jamais par un nouvel appel au modèle.

> Ces constats, comme celui de l'étape 5 sur la diffusion « réseaux », sont consignés avec leur étape de correction dans `docs/contexte-projet.md` (section 9, « Constats ouverts »), relue au début de chaque étape.

Les tests

39 nouveaux tests, qui tournent sans Ollama grâce à un client scripté :

- **Schéma** : catégories fermées, destinataire imposé, 8 réponses invalides refusées.
- **Routeur** : modèle de chaque palier, configuration C2, délai dépassé.
- **Graine** : stable entre deux processus.
- **Agent LLM** : réponse valide, relance avec la raison du refus, repli après deux échecs, délai dépassé puis succès, modèle injoignable.
- **Règles** : règle de repli, seuil borné et reproductible, porteur selon l'action.
- **Prompts** : marque lue dans la configuration, limite des messages montrés.
- **Orchestrateur** : publication des incidents et bilan des coûts.

Trois tests d'intégration (`-m ollama`) font décider un agent par palier sur les vrais modèles.

Résultat : 126 tests réussis, aucun ignoré, aucune remarque de ruff.

À retenir pour l'oral

- La forme de la réponse est garantie par une contrainte de génération, puis revalidée : le sens ne peut pas sortir des quatre catégories.
- Un modèle qui échoue ne bloque jamais le monde : une relance, puis une règle. Chaque incident est publié, compté et rattaché à sa décision.
- Chaque décision connaît son coût, tentatives ratées comprises ; le bilan le donne par modèle.
- Changer le modèle d'un palier est un simple paramètre : c'est ce qui rend les expériences C2 et C3 possibles.

Prochaine étape

Étape 7 — Tranche verticale : 3 agents LLM (un par palier) et 1 agent à règles sur 5 jours, de bout en bout par Kafka et Redis, lus en direct par un client console.

Étape 7 — Première tranche verticale

18 octobre 2026 · Exigences concernées : A2 (échanges entre agents), A4, A5, C2 (population fixée au lancement), C3 (deux horloges), E4 (coûts), F3 (Kafka et Redis).

Objectif

Faire tourner pour la première fois toute la chaîne avec de vrais modèles : orchestrateur, agents LLM des trois paliers, un agent à règles, Kafka, Redis, et un **client console** qui affiche en direct ce qui passe sur le bus. C'est la preuve que les briques des étapes 2 à 6 s'emboîtent.

Ce qui a été fait

Fichier	Rôle
`agents/population.py`	Construit la population depuis la configuration : agents LLM pour les rôles LLM, agent à seuil pour les rôles à règles, météo.
`agents/regles.py`	Ajout de l'agent à seuil, provisoire jusqu'aux vrais agents à règles (étape 13).
`bus/console.py`	Client console : lit les 7 topics et affiche chaque événement en une ligne lisible.
`sim/run.py`	Le run utilise désormais les agents LLM ; vérifie Ollama et les modèles avant de démarrer.
`configs/tranche.yaml`	Configuration de la tranche : 4 agents et la météo, 5 jours.

La population de la tranche

Agent	Moteur
`pharmacien_01`	qwen3:8b (grand)
`createur_contenu_01`	qwen3:4b (moyen)
`baigneur_occasionnel_01`	qwen3:1.7b (petit)
`fidele_autre_marque_01`	règle de seuil (seuil tiré : 67 %)
`meteo`	acteur du monde (factice jusqu'à l'étape 13)

Agents factices et agents réels partagent la même fonction d'identité (zone de départ, contacts, générateur aléatoire) : un run factice et un run LLM de même graine décrivent le même monde, seul le moteur de décision change. Un test le vérifie.

Comment le lancer

```
# terminal 1 : le client console attend le prochain run
.venv\Scripts\python -m bus.console

# terminal 2 : le run
.venv\Scripts\python -m sim.run --config configs/tranche.yaml
```

Extrait de ce qu'affiche la console :

```
──────── Jour 2 ────────
J2 · 00:00  météo       nuageux, 25.7 °C, UV 8
J2 · 10:00  décision    baigneur_occasionnel_01 (qwen3:1.7b, 0.7 s, 87 tok)
                  efficace · appliquer · porteur — « … Le pharmacien mentionne que ce
                  pourrait être efficace, mais il faut être prudent. Je vais peut-être
                  l'appliquer pour voir. »
J2 · 10:00  décision    fidele_autre_marque_01 [règle]
                  banal · ignorer · non porteur — « Règle de seuil : 0% de porteurs autour
                  de lui, sous son seuil de 67%. »
```

La console est un **observateur** : elle n'écrit jamais sur le bus, et un message non conforme y est signalé sans être renvoyé dans `sim.dlq`. Sinon, chaque console ouverte dupliquerait les rejets. Elle lit sans groupe de consommateurs Kafka, ce qui ne gêne aucun autre lecteur. La sortie complète du run est conservée dans `docs/mesures/tranche_2026-10-18_1142.md`.

Résultats du run de 5 jours

Mesure	Valeur
Décisions	16, dont 4 conservées sans appel (règle 10)
Appels LLM	11 (8b : 3, 4b : 3, 1.7b : 5), 0 relance, 0 repli, 0 incident
Messages	6, dont 5 sur les réseaux
Tokens	6 445 en entrée, 1 352 en sortie
Temps	64,6 s au total : 18,0 s d'appels, **46,6 s de chargement de modèles** (5 bascules)

On voit l'influence circuler : le pharmacien publie le jour 1 que la crème « tiendra peut-être la route » ; le jour 2, le baigneur cite « le pharmacien » et passe à *efficace* et *appliquer*. Le fidèle, seul dans sa zone sans porteur, ne bouge pas.

> **Constats de la tranche**, ajoutés à la table des constats ouverts (`docs/contexte-projet.md` , section 9) :
>
> - **Observations inventées.** Le jour 1, le pharmacien dit avoir « vu createur_contenu_01 parler de la marque sur les réseaux » alors qu'il n'a encore rien reçu. Ce n'est pas une violation de `caused_by` (rien n'y est cité), mais c'est une information du futur *dans le texte*. Étape 14 : détecter automatiquement un identifiant d'agent cité sans message reçu de lui, et le compter.
> - **Le temps de chargement domine** (72 % du run). Une bascule par jour, comme prévu à l'étape 5, mais d'environ 9 s : sur 90 jours, environ 14 min. À mesurer sur un run complet (étape 16).
> - **Personnages encore flous.** Le pharmacien parle comme un particulier (« c'est pas mon truc ») et reste *tendance* tous les jours. Un message du créateur a été coupé net par la longueur maximale. Prompts par rôle à l'étape 11.
> - **Météo factice incohérente** (« canicule » à UV 6, « nuageux » à UV 8) : vraie météo à règles à l'étape 13.

Une remarque technique

Au premier lancement, la console a attendu quelques secondes : c'était le tout premier lecteur à se connecter au broker avec un groupe, et Kafka créait son topic interne de suivi (`__consumer_offsets`). La console lit désormais sans groupe, ce qui supprime cette dépendance.

Les tests

4 nouveaux tests :

- la configuration de la tranche (un agent LLM par palier et un agent à règles) ;
- la population complète (bon moteur par rôle, seuils bornés, reproductible, mêmes identités que les agents factices) ;
- l'affichage de chaque type d'événement par la console ;
- un run de 2 jours **de bout en bout** sur les vrais Ollama et Kafka. Il vérifie que tout ce qui est publié est relu, que les appels relus égalent ceux du bilan, et que chaque décision vient du modèle de son palier.

Résultat : 130 tests réussis, aucun ignoré, aucune remarque de ruff.

À retenir pour l'oral

- La chaîne complète fonctionne : de l'horloge aux modèles, au bus, à l'état, à l'écran.
- Le même monde peut tourner avec des agents factices ou des agents LLM : on teste le moteur sans GPU, et on compare ensuite.
- Le coût d'un run se lit dans son bilan, par modèle ; ici, le chargement des modèles coûte plus que les appels eux-mêmes.
- Les premiers défauts de comportement des agents sont visibles et consignés, avec l'étape où ils seront traités.

Prochaine étape

Étape 8 — Historique et rejeu : écrire chaque run en Parquet, puis le relire à l'identique sans aucun appel LLM, avec les mêmes événements (E3, F5).

Étape 8 — Historique et rejeu sans LLM

18 octobre 2026 · Exigences concernées : E3 (rejeu à l'identique, sans appel LLM), F5 (historique interrogeable par des requêtes reproductibles), F3 (où vit chaque donnée), F6 (traçabilité).

Objectif

Garder chaque run en entier, pouvoir le **rejouer à l'identique sans aucun appel LLM** (même affichage, mêmes indicateurs) et interroger l'historique de tous les runs pour comparer des scénarios. C'est aussi le filet de la soutenance : si les modèles ne répondent pas le jour J, on rejoue un run enregistré.

Décisions prises à cette étape

Question	Décision	Raison
Qui écrit l'historique	Un **lecteur Kafka** lancé automatiquement par le run	On enregistre exactement ce qui a circulé sur le bus, pas ce que l'orchestrateur croit avoir publié. Une commande à part reconstruit l'historique depuis Kafka (rétention de 7 jours).
Par où passe le rejeu	**Directement depuis Parquet** vers un afficheur (console aujourd'hui, pont vers Godot à l'étape 9)	Les événements sont rendus à l'octet près et ne se mélangent jamais avec un run en direct.
Format	Parquet par lots (`runs/<run_id>/evenements/part-NNNNN.parquet`), lus avec DuckDB	Un arrêt brutal ne perd que le lot en cours. DuckDB lit tous les fichiers de tous les runs d'un coup, sans base à alimenter.
Ordre exact entre topics	Deux en-têtes Kafka posés par le producteur : `producteur` et `seq` (numéro de publication)	Chaque topic garde son ordre, mais pas l'ordre entre topics. L'horodatage est en millisecondes et ne départage pas des événements publiés dans la même milliseconde. Ces en-têtes ne changent pas le contrat d'événements.

Ce qui a été fait

Fichier	Rôle
`replay/ecrivain.py`	Écrivain d'historique : lit les 7 topics et écrit les événements du run en Parquet.
`replay/rejeu.py`	Relit un run dans l'ordre de publication, à la vitesse choisie, et vérifie son bilan.
`replay/bilan.py`	Recalcule le bilan d'un run à partir de ses seuls événements.
`replay/historique.py`	Vues DuckDB sur tous les runs et requêtes nommées.
`bus/producer.py`	En-têtes d'ordre de publication, et clé de tri partagée par la console et le rejeu.
`sim/run.py`	Démarre l'écrivain avant le premier événement et attend qu'il ait tout reçu.

Ce que contient l'historique

Une ligne par événement. La colonne `brut` garde l'événement exactement tel qu'il a circulé, et c'est elle qui sert au rejeu. Les autres colonnes reprennent l'enveloppe (type, dates simulées, jour, agent, zone, `caused_by`) et la position Kafka (topic, offset, horodatage, producteur, numéro de publication), pour les requêtes. En fin de run, le bilan indique combien d'événements ont été publiés et combien ont été enregistrés : les deux nombres doivent être égaux.

Le rejeu (E3)

```
.venv\Scripts\python -m replay.rejeu <run_id>              # tout, sans attente
.venv\Scripts\python -m replay.rejeu <run_id> --vitesse 2  # 2 jours simulés par seconde
.venv\Scripts\python -m replay.rejeu <run_id> --verifier  # bilan recalculé = bilan.json ?
```

Le rejeu affiche les événements avec la même fonction que la console en direct. Pour le contrôle `--verifier`, le bilan est recalculé avec la même classe `Bilan` que l'orchestrateur et dans le même ordre, si bien que même les sommes de durées tombent juste au millième près. Seuls les temps de la machine (durée totale, chargements, bascules) ne sont pas comparés : ils décrivent l'ordinateur, pas le monde simulé.

La preuve sur un vrai run LLM

Tranche verticale relancée (`20261018-121942-tranche-s42`), avec la console ouverte pendant le run, puis rejouée :

Vérification	Résultat
Événements publiés / enregistrés	18 / 18
Affichage en direct et affichage du rejeu	**30 lignes identiques**
Bilan recalculé depuis l'historique	**identique** à `bilan.json`
Appels LLM pendant le rejeu	0

> **Pourquoi le rejeu doit passer par le journal.** Ce run a la même graine que celui de l'étape 7, et il est pourtant différent : 7 décisions et 1 message, contre 16 et 6. Le pharmacien (8b) a donné mot pour mot la même réponse le jour 1, mais pas le baigneur ni le créateur. C'est ce que l'étape 6 annonçait : une graine ne rend pas un run LLM rejouable. On le rejoue donc depuis son enregistrement, jamais en rappelant les modèles. Ce constat est réglé et retiré de la table.

Deux défauts trouvés en vérifiant

- **Une course à la fin du run** : quand l'écrivain avait déjà tout reçu, la fin de run rendait la main avant l'écriture du dernier lot, et le bilan annonçait 0 événement enregistré. La fin de run attend maintenant que le dernier fichier soit écrit.
- **La console en direct n'affichait pas tout à fait l'ordre réel.** En reconstruisant l'historique de la tranche de l'étape 7 depuis Kafka, une ligne du rejeu n'était pas au même endroit qu'à l'écran. C'est la console qui avait tort : elle triait par heure de création des événements, et non par ordre de publication. Elle utilise désormais la même clé de tri que le rejeu.

L'historique interrogeable (F5)

Les vues `evenements`, `runs`, `decisions`, `messages` et `incidents` lisent directement les fichiers. Les champs des décisions (sens, action, modèle, tokens…) sont extraits du JSON brut, donc exactement de ce qui a circulé. Cinq requêtes nommées servent au rapport : `runs`, `sens_par_jour`, `cout_par_modele`, `incidents_par_nature`, `messages_par_palier`.

```
.venv\Scripts\python -m replay.historique --requete cout_par_modele
.venv\Scripts\python -m replay.historique --sql "SELECT palier, count(*) FROM decisions GROUP BY 1"
```

La requête `messages_par_palier` servira à mesurer le constat de l'étape 6 (propension à parler selon la taille du modèle).

Nouveau constat

> **Le monde se fige faute de déclencheur.** Dans ce run, à partir du jour 3, plus aucun agent ne décide : sans message reçu ni perturbation, chacun garde sa décision (règle 10), même quand la météo change. Le tableau des acteurs prévoit d'autres déclencheurs (exposition à un porteur, tendance perçue, publication planifiée, bilan hebdomadaire). À traiter aux étapes 11 et 13, puis à surveiller dans les bilans (part de décisions conservées).

Les tests

9 nouveaux tests :

- le bilan reconstruit depuis les seuls événements est égal au bilan tenu en direct, sur un run qui comporte relances, replis, un agent en panne et une décision mal formée ;
- l'historique Parquet redonne les événements à l'octet près et dans l'ordre de publication, même quand ils arrivent topic par topic dans la même milliseconde ;

- un bilan altéré est détecté, et un historique absent est signalé ;
- la vitesse du rejeu est respectée, et l'ordre de publication est bien départagé ;
- les requêtes DuckDB sur deux runs donnent les bons totaux, sont reproductibles, et aucun incident ne cite une décision absente (F6) ; elles fonctionnent aussi sans aucun historique ;
- de bout en bout sur Kafka : tout ce qui est publié est enregistré, et le rejeu redonne les mêmes événements et le même bilan.

Résultat : **139 tests réussis**, aucun ignoré, aucune remarque de ruff. Dépendances ajoutées : `pyarrow 26.0.0` et `duckdb 1.5.6`.

À retenir pour l'oral

- Un run se rejoue à l'identique sans appel LLM : même affichage ligne à ligne, même bilan, vérifiés automatiquement.
- Ce qui est enregistré, c'est ce qui a circulé sur le bus : l'écrivain est un consommateur Kafka comme les autres.
- Une graine ne suffit pas à reproduire un run LLM : la preuve est dans deux runs de même graine. Le journal est la seule source de vérité.
- Tous les runs s'interrogent ensemble en SQL, avec des requêtes nommées pour le rapport.

Prochaine étape

Étape 9 — Pont FastAPI : diffuser les événements vers Godot par WebSocket, en direct ou en rejeu, inspecter un agent (D2), injecter un événement (D3), mettre en pause ou arrêter un run.

Étape 9 — Pont FastAPI

18 octobre 2026 · Exigences concernées : D1 (affichage en direct), D2 (inspection d'un agent), D3 (injection d'un événement en direct), E3 (rejeu), C3 et C6 (dates et cohérence temporelle), F6 (traçabilité).

Objectif

Ouvrir la simulation à une interface. Godot (étape 10) n'a pas de client Kafka : le pont reçoit les événements et les pousse par WebSocket, en direct ou en rejeu. Il répond aussi aux questions de l'observateur (qui est cet agent, pourquoi a-t-il décidé cela ?) et transmet ses actions (injecter une perturbation, mettre en pause, arrêter).

Décisions prises à cette étape

Question	Décision	Raison
Où tourne le pont	Dans **son propre processus** (`python -m bridge`)	Il ne parle à la simulation qu'à travers Kafka, Redis et Parquet. Godot reste connecté pendant que les runs démarrent et s'arrêtent.
Quand une injection prend effet	Au **début du jour simulé suivant**	Cohérent avec la barrière synchrone et C6 : aucun agent ne découvre une perturbation au milieu d'une journée déjà commencée.
Qui la perçoit	Tout le monde, ou **une seule zone** si elle est précisée	Une rupture de stock à la pharmacie ne concerne pas la plage.
Quelles perturbations	Les 3 existantes (rupture de stock, polémique, vague de chaleur)	Leurs effets mécaniques arrivent avec les agents à règles (étape 13) ; la liste sera enrichie à l'étape 11.

Ce qui a été fait

Fichier	Rôle
`bridge/app.py`	L'application FastAPI : WebSocket, inspection, injections, commandes.
`bridge/vue.py`	Vue du monde construite à partir du seul flux d'événements (direct et rejeu).
`bridge/client.py`	Client de test du WebSocket, en attendant Godot.
`sim/controle.py`	Lecture de `sim.control` par l'orchestrateur.
`sim/orchestrateur.py`	Pause, reprise, arrêt propre, injection transformée en perturbation, ciblage par zone.
`sim/etat.py`	État Redis enrichi pour l'inspection, statut et heure simulée du run courant.
`docs/pont.md`	Référence du pont pour l'interface : routes, messages, codes de réponse.

Ce que propose le pont

Route	Exigence	Rôle
`WS /ws`	D1	Un instantané du monde à la connexion, puis chaque événement du run en cours.
`WS /ws?rejeu=<run_id>&vitesse=2`	E3	Le même flux, relu depuis l'historique : pour Godot, rien ne change.
`GET /agents/{id}`	D2	État de l'agent, justification et coût de sa dernière décision, ses 10 dernières décisions, ses messages. Pour un agent à règles : son seuil et la règle appliquée.
`POST /injections`	D3	Une perturbation, éventuellement limitée à une zone.
`POST /commandes`	—	Pause, reprise, arrêt, appliqués entre deux jours.
`GET /etat`, `/agents`, `/runs`	D1, E3	État courant, liste des agents, runs rejouables.

FastAPI génère une page `/docs` qui permet d'essayer chaque route à la main.

Le chemin d'une injection

1. L'observateur envoie `POST /injections`. Le pont publie un `controle.injection` sur `sim.control`, daté de l'heure simulée courante du run.
2. Avant le jour suivant, l'orchestrateur relève `sim.control` (un message non conforme part dans `sim.dlq`).
3. À 00:00 du jour suivant, il publie une `monde.perturbation` dont `caused_by` pointe sur l'injection (F6).
4. Les agents de la zone la perçoivent, ce qui les déclenche. Leurs décisions citent la perturbation, et leurs messages propagent l'effet.

La démonstration sur un vrai run LLM

Pont, client WebSocket et tranche de 5 jours. Au jour 1, injection d'une polémique sur les réseaux sociaux : « une vidéo virale accuse la crème de laisser des traces blanches et de coller au sable ». Trace complète dans `docs/mesures/pont_2026-10-18_1345.md`.

Jour	Ce qu'on voit dans le flux
J3, 00:00	La perturbation est publiée pour la zone *réseaux sociaux*.
J3	Le pharmacien, seul dans cette zone, passe de *tendance* à *dépassé* · *critiquer* : « les traces blanches c'est pas une option ici ». Il publie : « Trop de traces blanches, trop collante. »
J4	Le baigneur passe d'*efficace* à *banal* (« le pharmacien m'a dit qu'il ne l'utiliserait pas »). Le créateur *critique* à son tour, en citant `pharmacien_01`.

L'effet de l'injection se propage donc d'un agent à l'autre par les messages, ce que D3 demande. L'inspection du pharmacien montre sa justification, son coût (qwen3:8b, 579 tokens en entrée, 104 en sortie, 2,3 s) et une décision dont `caused_by` pointe sur la perturbation. Le bilan compte `injections : 1`, et l'injection du pont est enregistrée dans l'historique avec le reste (`evenements_externes : 1`).

Défauts trouvés en vérifiant

- **Heure simulée d'une injection.** Dans la démonstration, l'injection est datée J1 10:00 alors que la simulation était déjà au jour 2, d'où un effet au jour 3. Le pont lisait l'heure du dernier état écrit. L'orchestrateur publie maintenant son heure courante au début de chaque jour : l'injection est datée juste (C3), et prend effet au jour qui suit.
- **Comptage de l'historique.** L'écrivain enregistre aussi, à raison, les injections et commandes publiées par le pont pendant le run. Le contrôle de fin de run compare désormais les événements de l'orchestrateur et compte à part ceux du pont.

Nouveau constat

> **Une perturbation n'a encore qu'un effet narratif** : son texte est montré aux agents concernés, mais une rupture de stock n'empêche encore aucun achat et une vague de chaleur ne change pas la météo. Ces effets mécaniques viendront avec les agents à règles et la vraie météo (étape 13). Ajouté à la table des constats ouverts.

Les tests

17 nouveaux tests :

- **Orchestrateur** : une injection devient une perturbation le jour suivant, dans sa seule zone, datée et traçable, et recomptée par le rejeu. Pause puis reprise ; arrêt propre, y compris pendant une pause.
- **Vue et suivi** : la vue du monde est reconstruite depuis les événements ; le suivi en direct bascule sur un nouveau run et ignore les autres ; un client trop lent est déconnecté.
- **API** : injection publiée pour le run courant et datée de son heure simulée ; demandes invalides refusées (422) ; rien n'est publié sans run en cours (409) ; commandes ; inspection d'un agent LLM et d'un agent à règles ; instantané à la connexion ; rejeu par WebSocket identique à l'historique.

- **De bout en bout** : le vrai pont, sur Kafka et Redis, injecte une perturbation dans un vrai run puis l'arrête. La perturbation de l'historique cite bien l'injection, et le run s'arrête proprement avant sa fin.

Résultat : **156 tests réussis**, aucun ignoré, aucune remarque de ruff. Dépendances ajoutées : `fastapi 0.144.0`, `uvicorn 0.54.0`, `websockets 17.2`.

À retenir pour l'oral

- Le pont est indépendant de la simulation : il ne connaît que le bus, l'état et l'historique.
- Une injection suit le même chemin que tout le reste : un événement versionné, validé, daté dans les deux horloges et traçable jusqu'à ses effets.
- On a vu une polémique se propager : un agent la perçoit, critique, et deux autres reprennent ses mots le lendemain.
- Direct et rejeu passent par le même WebSocket : la démonstration peut se faire sur un run enregistré si les modèles font défaut.

Prochaine étape

Étape 10 — Scène Godot 3D : plusieurs maquettes low-poly à comparer (bâtiments distincts par zone, plage avec sable et mer, personnages animés), puis la scène choisie branchée sur le pont, avec le panneau d'indicateurs et l'inspection d'un agent (D1, D2, D3).